10-Day Django Bootcamp: Game Key Management & Publisher Webhooks

Course Description

Build a production-ready API for selling game keys. Publishers upload games, users buy keys, and **the system automatically notifies publishers via webhook when a game key expires**. Learn Django, DRF, async webhooks, and testing in 10 days (1.5h/day).

Prerequisites

- Basic Python, functions, classes, OOP
- HTTP basics, JSON

Tech Stack

Tool	Purpose
Python 3.12+	Runtime
`uv`	Package manager
Django 6.0	Web framework
DRF 3.17+	REST API
`python-decouple`	Config via `.env`
Celery + Redis	Async task queue
`pytest-django`	Testing
SQLite	Dev database

Deliverables

- GitHub repository with code, `.env` example, and README
- Working API: games, orders, key expiry detection
- Async webhook to publisher with retries and delivery logs
- pytest test suite (≥70% coverage on core)

Assessment

- Daily checkpoints: **20%**
- Final project: **80%**

⌨ Daily Syllabus (1.5 hours each day)

Day 1 – Environment & Project Setup

Goals: Install tooling, scaffold the Django project, configure environment variables, run the dev server.

```
# Install uv
curl -LsSf https://astral.sh/uv/install.sh | sh

# Create and activate virtual environment
uv venv
source .venv/bin/activate

# Add dependencies
uv add django djangorestframework python-decouple celery redis pytest-d

# Scaffold project and app
django-admin startproject gamekey_platform .
python manage.py startapp games
```

Create `.env` in the project root:

```
SECRET_KEY=your-secret-key-here
DEBUG=True
```

Update `gamekey_platform/settings.py` to use `python-decouple`:

```
from decouple import config

SECRET_KEY = config('SECRET_KEY')
DEBUG = config('DEBUG', default=False, cast=bool)

# Verify setup
python manage.py runserver
```

- Git init, add `.gitignore` (include `.env`)

🎓 Instructor Teaching Plan

⏱️ Time Breakdown (90 min)

Segment	Duration	Activity
Bootcamp intro & big picture	15 min	Diagram: what we're building end-to-end. Show the finished flow: user buys key → key expires → Celery fires webhook → publisher receives it. Make students care about the outcome before writing a single line.
Python ecosystem warmup	10 min	Briefly contrast `pip + venv` with `uv`. Why `uv`? Speed, lockfiles, reproducibility. Don't over-explain — run it live.
Live setup (students follow along)	25 min	Walk through every command in the Day 1 code block. Pause after `uv venv` and `source .venv/bin/activate` — confirm every student sees `(.venv)` in their prompt before continuing.

`python-decouple` and `.env`	15 min	Explain *why* secrets don't go in source code. Show what happens if you hard-code `SECRET_KEY` and push to GitHub. Set up `.env`, read it via `decouple`, add `.env` to `.gitignore`.
Git init & first commit	10 min	`git init`, stage, commit. Confirm `.env` is not tracked (`git status`).
Q&A + checkpoint verification	15 min	Every student must show the Django rocket page at `localhost:8000` before leaving.

📡 Key Concepts to Introduce

- **What is Django?** MTV framework (Model-Template-View). We'll use it purely as an API backend — no templates.
- **What is DRF?** A toolkit on top of Django for building REST APIs. Preview: it gives us serializers, viewsets, routers.
- **Virtual environments** — why isolation matters (dependency conflicts, reproducibility).
- **`DEBUG=True` vs `False`** — Django shows a detailed error page in debug mode. Always `False` in production.
- **Why `python-decouple`?** `os.environ.get()` works too, but `decouple` handles type casting, default values, and `.env` parsing in one clean API.

⚠️ Common Mistakes to Address

- **Forgetting to activate the venv** — the most common Day 1 error. Symptom: `django` not found even after `uv add`. Teach students to check `which python`.
- **Committing `.env` to git** — show *before* adding to `.gitignore` what `git status` reveals. Make the danger concrete.
- **`SECRET_KEY` still hard-coded** — students sometimes add `python-decouple` but forget to update `settings.py`. Verify by removing the `.env` line and confirming Django raises an error.
- **Windows path differences** — `.venv/Scripts/activate` not `.venv/bin/activate`. If any Windows users are present, flag this upfront.

❓ Check for Understanding

> "Why can't we just hard-code `SECRET_KEY = 'abc123'` in settings.py?"
>
> **Answer:** Hard-coding `SECRET_KEY` in source code poses a severe security risk. If the repository is pushed to a public platform (like GitHub), anyone can see the key. Attackers can use it to forge signed cookies, session data, or password reset tokens. Storing it in an environment variable (`.env` file) keeps it private and allows different values for development and production environments.
>
> "What would happen if two projects on the same machine installed different versions of `django` without virtual environments?"
>
> **Answer:** Installing different versions globally would cause a conflict because Python can only resolve one version of a package in its global site-packages directory. Installing a different version for the second project would overwrite the version needed by the first project, breaking it. Virtual environments

isolate dependencies per project, allowing each to run its required version independently.

"What does `DEBUG=True` change about how Django behaves?"

Answer: When `DEBUG=True`, Django displays detailed error traceback pages to the client (useful for debugging but a huge security leak in production because it exposes database queries, settings, and path variables). It also runs the built-in development server's auto-reloader and serves static/media files automatically. In production (`DEBUG=False`), it returns a generic 500 error page, requires `ALLOWED_HOSTS` to be set, and disables automated static file serving.

✓ **Day Checkpoint**

Student shows the Django welcome page at `http://127.0.0.1:8000/`. Their `.env` contains `SECRET_KEY` and `DEBUG`. Running `git log --oneline` shows at least one commit. `git status` shows `.env` as untracked (not staged).

Day 2 – Core Models: Publisher, Game, GameKey

Goals: Define the core data model and get it into the database.

`games/models.py`:

```
from django.db import models
from django.contrib.auth.models import User


class Publisher(models.Model):
    name = models.CharField(max_length=200)
    webhook_url = models.URLField()
    webhook_secret = models.CharField(max_length=64)
    user = models.OneToOneField(User, on_delete=models.CASCADE)

    def __str__(self):
        return self.name


class Game(models.Model):
    title = models.CharField(max_length=200)
    publisher = models.ForeignKey(Publisher, on_delete=models.CASCADE)
    price = models.DecimalField(max_digits=6, decimal_places=2)

    def __str__(self):
        return self.title


class GameKey(models.Model):
    STATUS_CHOICES = [('active', 'Active'), ('expired', 'Expired')]

    key_string = models.CharField(max_length=50, unique=True)
```

```
    game = models.ForeignKey(Game, on_delete=models.CASCADE)
    status = models.CharField(max_length=10, choices=STATUS_CHOICES, de
    expires_at = models.DateTimeField()
    owner = models.ForeignKey(User, on_delete=models.CASCADE, null=True

    def __str__(self):
        return self.key_string
```

```
python manage.py makemigrations
python manage.py migrate
```

Register all models in `games/admin.py`:

```
from django.contrib import admin
from .models import Publisher, Game, GameKey

admin.site.register(Publisher)
admin.site.register(Game)
admin.site.register(GameKey)
```

🎓 Instructor Teaching Plan

⏱️ Time Breakdown (90 min)

Segment	Duration	Activity
ORM mental model	20 min	Explain: Django models = Python classes = database tables. Draw the entity diagram: User ←→ Publisher ←→ Game ←→ GameKey. Discuss the relationships before writing code.
Write models live	25 min	Type out `Publisher`, `Game`, `GameKey` with students. Pause on each field type and explain the choice.
Migrations deep dive	15 min	Run `makemigrations`, open the generated file, read it aloud. Run `migrate`. Check with `python manage.py dbshell` or DB browser. Students should understand migrations are *versioned schema changes*, not magic.
Django admin	15 min	Register models, create a superuser, explore the admin. Create a sample Publisher and Game through the UI.
Q&A + checkpoint	15 min	Verify all 3 models visible and editable in admin.

📡 Key Concepts to Introduce

- **ORM = Object-Relational Mapper** — Python objects represent rows, class attributes represent columns.
- **`ForeignKey` vs `OneToOneField`** — FK: one publisher can have many games (one-to-many). O2O: one user can have at most one publisher profile.
- **`on_delete=CASCADE`** — deleting a Publisher deletes all their Games. Contrast with `SET_NULL` and `PROTECT`. Ask: "What should happen to GameKeys if we delete a Game?"
- **`null=True` vs `blank=True`** — `null` is database-level (allows NULL in column), `blank` is validation-level (allows empty string in forms/serializers). Common combo: `null=True, blank=True` for optional fields.

- **`choices`** — enforced at the Django layer, not at the database level. The DB stores the raw string `'active'/'expired'`.
- **Migrations** — two steps: `makemigrations` (create the migration file) → `migrate` (apply it to the DB). The migration file should be committed to git.

⚠ Common Mistakes to Address

- **Missing `__str__`** — the admin shows "Publisher object (1)" everywhere without it. Add `__str__` first, make it a habit.
- **Confusing `null` and `blank`** — students often add only `blank=True` for optional fields and then get database-level errors when saving.
- **Forgetting `'games'` in `INSTALLED_APPS`** — `makemigrations` silently produces no output and students are confused. Teach: always check `INSTALLED_APPS` first.
- **Running `migrate` without `makemigrations`** — nothing happens. Emphasize the two-step sequence.
- **Editing migrations by hand** — discourage this. Always re-run `makemigrations` after changing models.

? Check for Understanding

> "Why does `GameKey.owner` use `null=True, blank=True` but `GameKey.game` does not?"
>
> **Answer:** `GameKey.owner` is optional because a game key can exist (e.g., pre-loaded by a publisher) before any user purchases it. Thus, the database field must allow `NULL` (`null=True`) and API/form validation must allow it to be empty (`blank=True`). Conversely, a `GameKey` must always belong to a specific game, so `GameKey.game` is mandatory and cannot be null.
>
> "What's the difference between `makemigrations` and `migrate`? What does each file represent?"
>
> **Answer:** `makemigrations` scans model definitions for changes and creates new, numbered migration files (e.g., `0001_initial.py`), which are blueprints describing *how* to modify the database schema. `migrate` actually executes those blueprints against the database, applying the changes to create/modify tables. The migration files represent versioned history of the schema and must be committed to git.
>
> "If we delete a `Publisher`, what happens to their `Game` records? What about their `GameKey` records?"
>
> **Answer:** Because `Game.publisher` uses `on_delete=models.CASCADE`, deleting a `Publisher` automatically cascades and deletes all related `Game` records. In turn, because `GameKey.game` also uses `on_delete=models.CASCADE`, deleting those `Game` records will automatically cascade and delete all associated `GameKey` records.
>
> "Why is `key_string` marked `unique=True`?"
>
> **Answer:** A game key represents a single, unique digital asset. If it were not marked `unique=True`, the database could accidentally store duplicate key strings, leading to the same key being issued to multiple users, violating

licensing rules and causing validation collisions.

✓ **Day Checkpoint**

`python manage.py migrate` completes with no errors. Student navigates to `/admin`, logs in, and can create a `Publisher`, a `Game` linked to it, and a `GameKey` linked to the game — all through the admin UI.

Day 3 – DRF Basics: Game & Publisher APIs

Goals: Expose the models via REST endpoints using DRF ViewSets and a router.

Add to `INSTALLED_APPS` in `settings.py`:

```
INSTALLED_APPS = [
    ...
    'rest_framework',
    'games',
]
```

games/serializers.py:

```
from rest_framework import serializers
from .models import Game, Publisher


class GameSerializer(serializers.ModelSerializer):
    class Meta:
        model = Game
        fields = '__all__'


class PublisherSerializer(serializers.ModelSerializer):
    class Meta:
        model = Publisher
        exclude = ('webhook_secret',)  # never expose the secret
```

games/viewsets.py:

```
from rest_framework import viewsets
from .models import Game, Publisher
from .serializers import GameSerializer, PublisherSerializer


class GameViewSet(viewsets.ModelViewSet):
    queryset = Game.objects.all()
    serializer_class = GameSerializer


class PublisherViewSet(viewsets.ModelViewSet):
    queryset = Publisher.objects.all()
    serializer_class = PublisherSerializer
```

gamekey_platform/urls.py:

```
from django.contrib import admin
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from games.viewsets import GameViewSet, PublisherViewSet

router = DefaultRouter()
router.register(r'games', GameViewSet)
router.register(r'publishers', PublisherViewSet)

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include(router.urls)),
]
```

🎓 Instructor Teaching Plan

⏱ Time Breakdown (90 min)

Segment	Duration	Activity
REST & DRF overview	15 min	What is REST? Resources, HTTP verbs (GET/POST/PATCH/DELETE), status codes. What does DRF add on top of Django? Serializers for data validation + transformation, ViewSets for resource operations, Routers for automatic URL generation.
Serializers	20 min	Write `GameSerializer` and `PublisherSerializer`. Explain *why* we exclude `webhook_secret` from the publisher serializer — never expose secrets in API responses. Show `fields = '__all__'` first, then demonstrate the risk.
ViewSets	20 min	Write `GameViewSet` and `PublisherViewSet`. Compare ViewSet to a regular Django view: a ViewSet is one class that handles list, create, retrieve, update, destroy automatically.
Router & URLs	15 min	Register ViewSets with the router. Show what URLs the router generates (`/api/games/`, `/api/games/{id}/`). Browse the DRF web UI.
Live testing	10 min	Use curl or Postman/httpie to hit `GET /api/games/`, `POST /api/games/`, `GET /api/games/1/`. Read responses together.
Q&A + checkpoint	10 min	Verify browsable API works.

💡 Key Concepts to Introduce

- **Serializer = translator** — converts a Django model instance into JSON (serialization) and JSON into a validated Python dict (deserialization). Not just for output — serializers also validate incoming data.
- **`ModelSerializer`** — the shortcut. Reads the model's fields and generates the serializer automatically. `fields = '__all__'` includes every field; `exclude = (...)` removes specific ones.
- **ViewSet vs APIView** — `APIView` maps one HTTP method to one method (`get`, `post`). A `ViewSet` maps the full CRUD lifecycle to one class (`list`, `create`,

`retrieve`, `update`, `destroy`). The router wires these to URLs.

- **`DefaultRouter`** — generates `/api/games/` (list + create) and `/api/games/{pk}/` (retrieve + update + delete) automatically. Also adds a browsable root at `/api/`.
- **DRF browsable API** — great for development; shows a web form for POST requests. In production, disable it with `'DEFAULT_RENDERER_CLASSES': ['rest_framework.renderers.JSONRenderer']`.

⚠ Common Mistakes to Address

- **`'rest_framework'` not in `INSTALLED_APPS`** — the browsable API won't load, static CSS is missing. Common symptom: `DisallowedHost` or ugly unstyled pages.
- **Exposing sensitive fields** — students use `fields = '__all__'` on `Publisher` and wonder why `webhook_secret` appears in the response. This is a teaching moment: always be explicit about what you expose.
- **Forgetting to wire `include(router.urls)`** — `router.urls` is a list of URL patterns; it must be included in `urlpatterns`.
- **ViewSet queryset not set** — if `queryset` is missing, DRF raises `AssertionError` about `basename`. Always set `queryset` or override `get_queryset()`.

? Check for Understanding

"What does a serializer do with incoming data before it reaches the database?"

Answer: A serializer validates the structure and types of incoming data against defined field rules, checks for required fields, runs custom validation logic (e.g., checking if a value is valid), and parses the raw JSON input into a validated Python dictionary (`serializer.validated_data`).

"What's the difference between `GET /api/games/` and `GET /api/games/1/`? Which ViewSet methods handle each?"

Answer: `GET /api/games/` retrieves a list of all games and is handled by the `list` method of `GameViewSet`. `GET /api/games/1/` retrieves the details of a single game with ID 1 and is handled by the `retrieve` method of `GameViewSet`.

"Why might you want `fields = ('id', 'title', 'price')` instead of `'__all__'`?"

Answer: Explicitly defining fields improves security and API efficiency by ensuring that sensitive internal fields (like database flags, internal statuses, or relationship keys) are not leaked to the frontend, and reduces bandwidth by excluding unused fields.

"What would happen if we forgot to exclude `webhook_secret` from the publisher serializer?"

Answer: The `webhook_secret` would be serialized and returned in public GET/POST responses. This would allow anyone viewing the API output to obtain the secret, enabling them to forge webhook requests and make unauthorized deliveries appear authentic to the publisher's system.

✓ Day Checkpoint

`GET /api/games/` returns `200 OK` with a JSON array (possibly empty). `POST /api/games/` with a valid body creates a game and returns `201 Created`. Student confirms `webhook_secret` does not appear in `GET /api/publishers/` responses.

Day 4 – Authentication & Permissions

Goals: Lock down the API with token auth and owner-level permissions.

Add to `INSTALLED_APPS`:

```
'rest_framework.authtoken',
```

```
python manage.py migrate
```

Update `settings.py`:

```
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework.authentication.TokenAuthentication',
    ],
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticatedOrReadOnly',
    ],
}
```

`games/permissions.py` — custom permission so publishers can only edit their own games:

```
from rest_framework.permissions import BasePermission, SAFE_METHODS


class IsOwnerOrReadOnly(BasePermission):
    def has_object_permission(self, request, view, obj):
        if request.method in SAFE_METHODS:
            return True
        return obj.publisher.user == request.user
```

Add a user registration endpoint in `games/views.py`:

```
from django.contrib.auth.models import User
from rest_framework.authtoken.models import Token
from rest_framework.decorators import api_view, permission_classes
from rest_framework.permissions import AllowAny
from rest_framework.response import Response
from rest_framework import status


@api_view(['POST'])
@permission_classes([AllowAny])
def register(request):
    username = request.data.get('username')
    password = request.data.get('password')
    if not username or not password:
```

```
        return Response({'error': 'Username and password required.'}, s
    user = User.objects.create_user(username=username, password=passwor
    token, _ = Token.objects.get_or_create(user=user)
    return Response({'token': token.key}, status=status.HTTP_201_CREATE
```

Add to `urls.py`:

```
from games.views import register
urlpatterns += [path('api/register/', register)]
```

🎓 Instructor Teaching Plan

⏱️ Time Breakdown (90 min)

Segment	Duration	Activity
Auth concepts	15 min	Compare session auth (cookie-based, browser), token auth (header-based, API), and JWT (self-contained). Explain why token auth is the right choice for a stateless API.
Token auth setup	15 min	Add `rest_framework.authtoken` to `INSTALLED_APPS`, run `migrate`, update `REST_FRAMEWORK` settings. Generate a token via `manage.py shell` to verify.
Custom permission class	20 min	Write `IsOwnerOrReadOnly`. Explain `has_permission` (request-level) vs `has_object_permission` (object-level). Explain `SAFE_METHODS`. Apply to `GameViewSet`.
Registration endpoint	20 min	Write the `register` view together. Explain `create_user` (hashes password) vs `create` (stores plaintext — never do this). Show `Token.objects.get_or_create`.
End-to-end auth test	10 min	Register a user → get token → use token in `Authorization: Token ...` header → hit a protected endpoint. Try without the header → confirm 401.
Q&A + checkpoint	10 min	

💡 Key Concepts to Introduce

- **Authentication vs Authorization** — authentication asks "who are you?", authorization asks "what are you allowed to do?". Token auth handles the first; permission classes handle the second.
- **`IsAuthenticatedOrReadOnly`** (global default) — unauthenticated users can `GET`; only authenticated users can `POST/PATCH/DELETE`. Good starting default for a public catalog.
- **`BasePermission`** — the base class for all DRF permissions. Override `has_permission` for request-level checks, `has_object_permission` for instance-level checks. Both must return `True` for access to proceed.
- **`SAFE_METHODS = ('GET', 'HEAD', 'OPTIONS')`** — read-only HTTP methods. The pattern "allow reads for all, writes only for owner" is idiomatic DRF.
- **`User.objects.create_user()`** — hashes the password via Django's password hasher. `create()` skips hashing. Always use `create_user` for real users.
- **`@permission_classes([AllowAny])`** — explicitly bypasses the global default for a specific view. Required for registration — the user doesn't have a token yet.

⚠ Common Mistakes to Address

- **Using `create()` instead of `create_user()`** — password stored in plaintext, authentication will always fail.
- **Missing `rest_framework.authtoken` migration** — `Token` model doesn't exist yet, Django raises `ProgrammingError`. Always run `migrate` after adding the app.
- **Wrong `Authorization` header format** — must be `Token <token>` with a space (not `Bearer`, not `token`). Confuses students coming from JWT backgrounds.
- **`has_object_permission` not called for list actions** — DRF only calls `has_object_permission` when the view retrieves a specific object (retrieve, update, destroy). It's never called for `list` or `create`.
- **`@permission_classes` not overriding the global default** — students add `@permission_classes([AllowAny])` but forget `@api_view(['POST'])`, so Django treats it as a regular function view and DRF never runs.

? Check for Understanding

> "What's the difference between `has_permission` and `has_object_permission`? Give an example where you'd use each."
>
> **Answer:** `has_permission` checks if the user has access to the endpoint in general (checked at the start of the request, e.g., "Is the user logged in?"). `has_object_permission` is only called during detail views (retrieve/update/delete) to check access to a specific database object (e.g., "Is the logged-in user the owner of this game record?").
>
> "Why do we use `create_user()` instead of `create()` when making User objects?"
>
> **Answer:** `create_user()` is a helper method on Django's User manager that handles password hashing using a secure algorithm (like PBKDF2). Using `create()` directly would write the password to the database as plaintext, exposing it to database administrators or security breaches, and preventing the user from logging in since Django's auth system expects a hashed password.
>
> "If we didn't add `@permission_classes([AllowAny])` to the register view, what would happen when a new user tries to register?"
>
> **Answer:** The registration request would be blocked by the global permission default (`IsAuthenticatedOrReadOnly`), returning a `401 Unauthorized` or `403 Forbidden` response. Since a new user does not have a token yet, they would be unable to register.
>
> "What does `IsAuthenticatedOrReadOnly` allow vs deny? Is it the right default for this API?"
>
> **Answer:** It allows anyone (authenticated or anonymous) to perform safe, read-only HTTP methods (`GET`, `HEAD`, `OPTIONS`), but restricts writing methods (`POST`, `PUT`, `PATCH`, `DELETE`) to authenticated users. It is an excellent default for this API because it keeps the game catalog publicly browsable while securing modifications and purchases.

✓ **Day Checkpoint**

`POST /api/register/` returns a token. `POST /api/games/` without the token returns `401`. `POST /api/games/` with `Authorization: Token <token>` returns `201`. `PATCH /api/games/1/` with a token from a *different* user returns `403`.

Day 5 – Orders API: Buying a Game Key

Goals: Build the purchase flow — atomically assign a key and set its expiry.

Add `Order` and `OrderItem` to `games/models.py`:

```
class Order(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    purchased_at = models.DateTimeField(auto_now_add=True)


class OrderItem(models.Model):
    order = models.ForeignKey(Order, on_delete=models.CASCADE)
    game_key = models.OneToOneField(GameKey, on_delete=models.CASCADE)
```

`games/views.py` — `POST /api/orders/`:

```
import uuid
from datetime import timedelta
from django.utils import timezone
from django.db import transaction
from rest_framework.decorators import api_view, permission_classes
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response
from rest_framework import status
from .models import Game, GameKey, Order, OrderItem


@api_view(['POST'])
@permission_classes([IsAuthenticated])
def create_order(request):
    game_id = request.data.get('game_id')
    if not game_id:
        return Response({'error': 'game_id is required.'}, status=statu

    try:
        game = Game.objects.get(id=game_id)
    except Game.DoesNotExist:
        return Response({'error': 'Game not found.'}, status=status.HTT

    with transaction.atomic():
        # Use select_for_update to prevent race conditions
        existing_key = (
            GameKey.objects.select_for_update()
            .filter(game=game, status='active', owner__isnull=True)
            .first()
```

```
        )

        if existing_key:
            # Assign existing unowned key
            key = existing_key
            key.owner = request.user
            key.save()
        else:
            # Generate a fresh key
            key = GameKey.objects.create(
                key_string=str(uuid.uuid4()).upper(),
                game=game,
                status='active',
                expires_at=timezone.now() + timedelta(days=30),
                owner=request.user,
            )

        order = Order.objects.create(user=request.user)
        OrderItem.objects.create(order=order, game_key=key)

    return Response({
        'order_id': order.id,
        'game': game.title,
        'key': key.key_string,
        'expires_at': key.expires_at,
    }, status=status.HTTP_201_CREATED)
```

Wire URL: `path('api/orders/', create_order)` in `urls.py`.

🎓 Instructor Teaching Plan

⏱ Time Breakdown (90 min)

Segment	Duration	Activity
Race condition motivation	20 min	Before writing any code, pose the problem: two users hit POST /api/orders/ at the exact same millisecond for the same game. Without locks, they both read the same available key and both get assigned it — a key is sold twice. Draw the timeline on a whiteboard.
Add Order & OrderItem models	15 min	Add models, run `makemigrations` + `migrate`. Register in admin.
Walk through `create_order` view	30 min	Go line by line. Emphasize `transaction.atomic()`, `select_for_update()`, the `owner__isnull=True` filter. Explain why we generate a fresh key if no pre-loaded keys exist.
Wire URL & live test	15 min	Add URL, test with Postman. Create a game through admin, then buy it. Show the key and expiry in the response.
Q&A + checkpoint	10 min	

🌤 Key Concepts to Introduce

- **`transaction.atomic()`** — everything inside the `with` block executes as one DB transaction. If any exception is raised, all changes are rolled back.
- **`select_for_update()`** — places a row-level lock on the selected rows. Any other transaction trying to lock the same rows will wait until this transaction completes. This is *pessimistic locking*.
- **Race condition** — a bug that depends on the timing of concurrent operations. Hard to reproduce in development (single user), catastrophic in production (many users).
- **`auto_now_add=True`** on `Order.purchased_at` — automatically sets to `timezone.now()` on creation. Read-only after creation. Compare with `auto_now=True` (updates on every save).
- **`uuid4()`** for key generation — cryptographically random, collision probability negligible. Better than auto-increment integers for keys that users will see.
- **`timedelta(days=30)`** — adding a duration to a datetime. Explain `timezone.now()` vs `datetime.now()` — always use `timezone.now()` in Django to stay timezone-aware.

⚠ Common Mistakes to Address

- **Not using `transaction.atomic()`** — the key is locked but if an exception happens after the lock, the key status is left in an inconsistent state.
- **Filtering with `status='active'` but not `owner__isnull=True`** — would return keys that are already owned by someone else.
- **Not capturing keys before `update()`** — in Day 6/8, students will update the queryset with `.update(status='expired')`, which clears the queryset. Foreshadow: always materialize (`list()`) before bulk-updating.
- **Returning the raw `key_string` field vs a formatted version** — fine for now, but flag that in production you'd never expose this in a GET endpoint (only return once, at purchase time).
- **Missing `@permission_classes([IsAuthenticated])`** — anyone could buy keys without registering.

? Check for Understanding

> "What would go wrong without `select_for_update()`?"
>
> **Answer:** Without `select_for_update()`, a race condition could occur if two concurrent requests query the database at the same time for an available key. Both would find the same unowned key, assign it to their respective user, and write it back. This results in the same game key being sold twice (double-allocation).
>
> "When does `transaction.atomic()` roll back? What triggers a rollback?"
>
> **Answer:** It rolls back when an unhandled exception is raised within the context manager block. If any error (like a database constraint violation or a Python runtime exception) occurs before the block exits successfully, all changes made to the database within that block are discarded.
>
> "Why do we filter with `owner__isnull=True`? What does `owner` represent for a key that no one has purchased yet?"

Answer: We filter for `owner__isnull=True` to retrieve a key that has not yet been bought by any user. For an unpurchased key, `owner` is `None` (represented as `NULL` in the database). Filtering this way prevents re-allocating an already purchased key.

"Why use `uuid4()` to generate the key string instead of, say, `GameKey.objects.count() + 1`?"

Answer: `uuid4()` generates cryptographically random, unique identifiers that cannot be guessed by users, preventing attackers from predicting and stealing other game keys. Using `count() + 1` is sequential, exposing the total volume of keys and making them trivial to guess and exploit.

✓ **Day Checkpoint**

`POST /api/orders/` with a valid `game_id` and auth token returns `201` with `order_id`, `game`, `key`, and `expires_at`. `POST /api/orders/` without a token returns `401`. `POST /api/orders/` with a non-existent `game_id` returns `404`.

Day 6 – Detecting Expired Keys (Management Command)

Goals: Write a management command that marks expired keys and can be scheduled via cron.

Create the file structure:

```
games/
  management/
    __init__.py
    commands/
      __init__.py
      check_expired_keys.py
```

games/management/commands/check_expired_keys.py:

```
from django.core.management.base import BaseCommand
from django.utils import timezone
from games.models import GameKey


class Command(BaseCommand):
    help = 'Mark active keys whose expiry has passed as expired.'

    def handle(self, *args, **options):
        expired_keys = GameKey.objects.filter(
            expires_at__lte=timezone.now(),
            status='active'
        )
        count = expired_keys.update(status='expired')
        self.stdout.write(self.style.SUCCESS(f'Expired {count} keys.'))

# Test manually
```

```
python manage.py check_expired_keys
```

To schedule in production, add to crontab:

```
*/15 * * * * /path/to/.venv/bin/python /path/to/manage.py check_expired
```

🎓 Instructor Teaching Plan

⏱ Time Breakdown (90 min)

Segment	Duration	Activity
Management commands overview	15 min	What are management commands? When are they the right tool vs. a scheduled job vs. a cron script vs. a Celery task? Examples: `migrate`, `collectstatic`, `createsuperuser`. These are Django's built-in admin scripts — we can write our own.
File structure creation	10 min	Create `management/`, `management/__init__.py`, `management/commands/`, `management/commands/__init__.py`. Emphasize: both `__init__.py` files are required. Common mistake: missing one of them.
Write the command	25 min	Walk through `BaseCommand`, `help`, `handle()`, `self.stdout.write()`, `self.style.SUCCESS()`. Explain `.update()` as a bulk DB operation vs looping and calling `.save()`.
Test manually	15 min	Create a `GameKey` in admin with `expires_at` in the past. Run `python manage.py check_expired_keys`. Verify status changes in admin.
Cron scheduling	15 min	Show the cron syntax. Discuss: cron runs on one server — what if the server goes down? (Preview: Celery Beat solves this, but cron is fine for the bootcamp scope.)
Q&A + checkpoint	10 min	

💡 Key Concepts to Introduce

- **`BaseCommand`** — the base class for all management commands. `handle()` is the entry point; `help` is the description shown in `manage.py help`.
- **`self.stdout.write()` vs `print()`** — management commands should use `self.stdout` so the output can be captured and redirected (e.g., in tests via `call_command`).
- **`self.style.SUCCESS()` / `.ERROR()` / `.WARNING()`** — colorizes terminal output. Only applies when running in a terminal (not captured output).
- **Bulk update: `.update()`** — a single `UPDATE ... WHERE ...` SQL statement. Far more efficient than loading each object into Python and calling `.save()`. Trade-off: `save()` triggers model signals; `.update()` does not.
- **Timezone-aware datetimes** — `timezone.now()` returns a timezone-aware datetime. If `expires_at` were timezone-naive, the `__lte` comparison would fail. Always use `timezone.now()` in Django.
- **`expires_at__lte=timezone.now()`** — Django ORM field lookup: "less than or equal to now", i.e., "already expired".

⚠ Common Mistakes to Address

- **Missing `__init__.py` files** — Django won't discover the command. The error is cryptic (`Unknown command: 'check_expired_keys'`). Teach students to check the directory structure first.
- **Using `.save()` in a loop** — works but generates N SQL queries. For 1000 expired keys, that's 1000 UPDATE statements vs one. Make the performance case.
- **Timezone-naive comparisons** — if a student uses `datetime.now()` (from the `datetime` module, not Django's `timezone`), they get a `TypeError` when comparing with a timezone-aware `expires_at`.
- **The queryset-after-update problem** — after `expired_keys.update(status='expired')`, the queryset is "stale" — the objects in Python still have `status='active'`. If you iterate after `.update()`, you get the pre-update values. (This will be critical in Day 8.)

? Check for Understanding

> "Why do we use `self.stdout.write()` instead of `print()` inside a management command?"
>
> **Answer:** Using `self.stdout.write()` is best practice because it integrates with Django's internal logging systems and output redirection. In testing, it allows the output to be intercepted and asserted on (using `call_command`), whereas `print()` writes directly to the standard system stdout, making it harder to test or suppress.
>
> "What's the performance difference between `.update()` and looping + `.save()`? Are there situations where you'd prefer `.save()` anyway?"
>
> **Answer:** `.update()` executes a single SQL `UPDATE` statement in the database, which is extremely fast and efficient for bulk operations. Looping and calling `.save()` executes a separate SQL query for each record (N queries), which is slow. However, you would prefer `.save()` if you need to trigger model `save()` overrides, pre/post-save signals, or validation, which `.update()` bypasses.
>
> "What does `expires_at__lte=timezone.now()` translate to in SQL?"
>
> **Answer:** It translates to a `WHERE` clause: `WHERE expires_at <= 'CURRENT_TIMESTAMP_VALUE'`. This filters for records where the expiration datetime is less than or equal to the current time.
>
> "If we want this command to run every 15 minutes automatically, what are our options?"
>
> **Answer:** 1) Setup a system-level cron job that executes the command through the virtualenv python. 2) Use a Celery Beat task that runs the management command (or its logic) on a schedule. 3) Use cloud-specific schedulers (like Render Cron Jobs or AWS EventBridge) to trigger the command.

✓ Day Checkpoint

Student creates a `GameKey` in admin with `expires_at` set to one hour ago and

`status='active'`. They run `python manage.py check_expired_keys` and see the success message. Refreshing admin shows `status='expired'`.

Day 7 – Webhook Fundamentals (Synchronous)

Goals: Understand webhook mechanics — payload signing with HMAC, HTTP delivery, and the downsides of doing it synchronously.

Extend the management command with a synchronous webhook sender:

`games/webhooks.py`:

```
import hmac
import hashlib
import json
import requests


def send_expiry_webhook(publisher, game_key_str, game_title, expired_at
    payload = {
        "event": "game_key.expired",
        "game_key": game_key_str,
        "game_title": game_title,
        "expired_at": expired_at.isoformat(),
    }

    secret = publisher.webhook_secret.encode()
    body = json.dumps(payload, sort_keys=True).encode()
    signature = hmac.new(secret, body, hashlib.sha256).hexdigest()

    headers = {
        "Content-Type": "application/json",
        "X-Signature": f"sha256={signature}",
    }

    try:
        response = requests.post(publisher.webhook_url, json=payload, h
        response.raise_for_status()
    except Exception as e:
        print(f"[Webhook] Delivery failed for publisher {publisher.id}:
```

Updated `check_expired_keys.py` using sync delivery:

```
from django.core.management.base import BaseCommand
from django.utils import timezone
from games.models import GameKey
from games.webhooks import send_expiry_webhook


class Command(BaseCommand):
    help = 'Mark expired keys and notify publishers synchronously.'
```

```
    def handle(self, *args, **options):
        expired_keys = GameKey.objects.select_related('game__publisher'
            expires_at__lte=timezone.now(),
            status='active'
        )
        count = expired_keys.update(status='expired')

        for key in GameKey.objects.filter(status='expired').select_rela
            send_expiry_webhook(key.game.publisher, key.key_string, key

        self.stdout.write(f'Expired {count} keys (sync webhooks sent).'
```

Discussion points:

- Synchronous HTTP calls block the management command thread
- A slow or unavailable publisher endpoint delays all subsequent notifications
- No retry logic — a failed delivery is silently lost
- → Solution: move webhook delivery to an async task queue (Day 8)

🎓 Instructor Teaching Plan

⏱️ Time Breakdown (90 min)

Segment	Duration	Activity
What is a webhook?	20 min	Contrast webhooks with polling. Real-world examples: GitHub fires a webhook when a PR is merged; Stripe fires one when a payment succeeds. Draw the pub/sub flow: publisher registers a URL → our system fires HTTP POST when something happens → publisher handles it.
HMAC signing	20 min	Why sign? Because anyone can POST to the publisher's webhook URL. HMAC lets the publisher verify the payload came from us and wasn't tampered with. Walk through the algorithm: `secret + payload → SHA256 digest`. Show `sort_keys=True` and why it matters for reproducibility.
Write `webhooks.py`	20 min	Code the synchronous sender together. Test with webhook.site — a free endpoint that logs every request. Students should see the payload arrive.
Update management command	10 min	Integrate sync sender. Run manually. Observe blocking behavior (if webhook.site is slow, the command waits).
Discussion: sync limitations	10 min	Go through the 3 bullet points at the bottom of Day 7. Collect student ideas on how to fix each. Preview: Day 8 solves all three with Celery.
Q&A + checkpoint	10 min	

📯 Key Concepts to Introduce

- **Webhook = HTTP callback** — instead of the subscriber polling "has anything changed?", the publisher pushes "something changed!" to a pre-registered URL. Push >

pull for latency and efficiency.

- **HMAC-SHA256** — Hash-based Message Authentication Code. Both parties share a secret. The sender computes `HMAC(secret, payload)` and sends it as a header. The receiver recomputes it and compares — if they match, the payload is authentic and untampered.
- **`sort_keys=True` in `json.dumps()`** — JSON object key order is not guaranteed. If the receiver serializes the same data differently (different key order), the signature won't match. `sort_keys=True` makes the serialization deterministic.
- **`hmac.compare_digest()`** — constant-time comparison, not `==`. Prevents timing attacks where an attacker could determine the correct signature byte by byte by measuring response time.
- **`requests.post(..., timeout=5)`** — always set a timeout on outbound HTTP calls. Without it, your process can hang forever if the publisher's server is slow or down.
- **`raise_for_status()`** — raises `HTTPError` for 4xx/5xx responses. Without it, a `500` from the publisher looks like success.

⚠ Common Mistakes to Address

- **`hmac.new()` vs `hmac.HMAC()`** — Python's `hmac` module uses `hmac.new()` (older API) or `hmac.HMAC()` (Python 3.10+). Both work; be consistent. Common bug: mixing them.
- **Not encoding strings to bytes** — `hmac.new(secret, body, ...)` requires `bytes`. `secret.encode()` and `body` (already bytes from `.encode()`) — forgetting either raises `TypeError`.
- **Not handling exceptions** — if `requests.post()` raises a `ConnectionError` (publisher server is down), the whole management command crashes without processing the other keys. Wrap in `try/except`.
- **No timeout on `requests.post()`** — if the publisher's server never responds, the management command hangs indefinitely. `timeout=5` is a reasonable default.
- **Reusing `payload` dict** — if you modify `payload` after signing (e.g., to add extra fields), the signature no longer matches. Serialize and sign at the same point.

? Check for Understanding

> "A publisher claims they received a webhook from us but the payload was tampered with. How does HMAC protect against this?"
>
> **Answer:** HMAC uses a shared secret to generate a cryptographic signature of the request body. If any character in the payload is altered during transit, the receiver's computed signature will not match the `X-Signature` header, revealing that the payload was modified and should be rejected.
>
> "Why do we pass `sort_keys=True` to `json.dumps()`? What could go wrong without it?"
>
> **Answer:** JSON keys are unordered. If we don't sort keys, the serialization format might change (e.g. dictionary keys serialized in a different order), resulting in a different byte sequence. This would produce a different HMAC signature, causing verification to fail even if the content remains identical.
>
> "What happens to the management command if the publisher's server is down

and we're making synchronous HTTP calls?"

Answer: The management command thread will block, waiting for the HTTP request to timeout (e.g. 5 seconds per request). If many keys expired for that publisher, or multiple publishers are down, the script will take a very long time to complete and could cause other pending updates to stall.

"What's the difference between a 4xx and 5xx response from the publisher's webhook endpoint? Should we retry both?"

Answer: A `4xx` response represents a client error (e.g., `400 Bad Request` or `404 Not Found`), suggesting our payload is invalid or the endpoint is misconfigured; retrying will likely fail again, so we shouldn't retry. A `5xx` response represents a server error (e.g., `500 Internal Server Error` or `503 Service Unavailable`), suggesting temporary publisher outage; we should retry these as the server might recover.

✓ **Day Checkpoint**

Student uses webhook.site as the publisher's `webhook_url`. After running `python manage.py check_expired_keys`, the webhook.site log shows the POST request with the correct JSON body and `X-Signature` header. The student can manually verify the HMAC in a Python shell.

Day 8 – Async Webhooks with Celery

Goals: Move webhook delivery off the main thread using Celery + Redis, add automatic retries, and log every delivery attempt.

Step 1 — Run Redis

```
docker run -d -p 6379:6379 redis:7-alpine
```

Step 2 — Configure Celery

`gamekey_platform/celery.py`:

```
import os
from celery import Celery

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'gamekey_platform.setti

app = Celery('gamekey_platform')
app.config_from_object('django.conf:settings', namespace='CELERY')
app.autodiscover_tasks()
```

Update `gamekey_platform/__init__.py` so Django loads Celery on startup:

```
from .celery import app as celery_app
```

```
__all__ = ('celery_app',)
```

Add to `settings.py`:

```
CELERY_BROKER_URL = 'redis://localhost:6379/0'
CELERY_RESULT_BACKEND = 'redis://localhost:6379/0'
CELERY_TASK_ALWAYS_EAGER = False  # set True in tests to run tasks sync
```

Step 3 — Webhook Delivery Log Model

Add to `games/models.py`:

```
class WebhookDeliveryLog(models.Model):
    game_key = models.CharField(max_length=50)
    publisher = models.ForeignKey(Publisher, on_delete=models.CASCADE)
    payload = models.JSONField()
    response_status = models.IntegerField(null=True, blank=True)
    error_message = models.TextField(blank=True)
    attempt = models.IntegerField(default=0)
    success = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ['-created_at']

    def __str__(self):
        status = 'OK' if self.success else 'FAIL'
        return f"[{status}] {self.game_key} attempt #{self.attempt}"
```

```
python manage.py makemigrations
python manage.py migrate
```

Register in `admin.py`:

```
from .models import WebhookDeliveryLog
admin.site.register(WebhookDeliveryLog)
```

Step 4 — Celery Task with Retries

`games/tasks.py`:

```
import hmac
import hashlib
import json
import requests
from celery import shared_task
from .models import Publisher, WebhookDeliveryLog


@shared_task(bind=True, max_retries=3, default_retry_delay=60)
```

```
def send_expiry_webhook_async(self, publisher_id, game_key_str, game_ti
    publisher = None
    payload = {}

    try:
        publisher = Publisher.objects.get(id=publisher_id)

        payload = {
            "event": "game_key.expired",
            "game_key": game_key_str,
            "game_title": game_title,
            "expired_at": expired_at_iso,
            "attempt": attempt,
        }

        secret = publisher.webhook_secret.encode()
        body = json.dumps(payload, sort_keys=True).encode()
        signature = hmac.new(secret, body, hashlib.sha256).hexdigest()

        headers = {
            "Content-Type": "application/json",
            "X-Signature": f"sha256={signature}",
        }

        response = requests.post(
            publisher.webhook_url,
            json=payload,
            headers=headers,
            timeout=5,
        )

        WebhookDeliveryLog.objects.create(
            game_key=game_key_str,
            publisher=publisher,
            payload=payload,
            response_status=response.status_code,
            attempt=attempt,
            success=response.status_code < 400,
        )

        if response.status_code >= 400:
            raise Exception(f"HTTP {response.status_code} from publishe

    except Exception as exc:
        if publisher:
            WebhookDeliveryLog.objects.create(
                game_key=game_key_str,
                publisher=publisher,
                payload=payload,
                error_message=str(exc),
                attempt=attempt,
                success=False,
```

```
            )

        # Exponential backoff: 60s, 120s, 240s
        raise self.retry(exc=exc, countdown=60 * (2 ** attempt))
```

Step 5 — Update Management Command

games/management/commands/check_expired_keys.py:

```
from django.core.management.base import BaseCommand
from django.utils import timezone
from games.models import GameKey
from games.tasks import send_expiry_webhook_async


class Command(BaseCommand):
    help = 'Mark expired keys and dispatch async webhook notifications.

    def handle(self, *args, **options):
        expired_keys = GameKey.objects.select_related('game__publisher'
            expires_at__lte=timezone.now(),
            status='active'
        )

        # Capture before update() clears the queryset
        keys_to_notify = list(expired_keys)
        count = expired_keys.update(status='expired')

        for key in keys_to_notify:
            send_expiry_webhook_async.delay(
                publisher_id=key.game.publisher.id,
                game_key_str=key.key_string,
                game_title=key.game.title,
                expired_at_iso=key.expires_at.isoformat(),
                attempt=0,
            )

        self.stdout.write(self.style.SUCCESS(
            f'Expired {count} keys. Webhook tasks dispatched to Celery.
        ))
```

Step 6 — Run the Celery Worker

```
celery -A gamekey_platform worker --loglevel=info
```

In a second terminal, run the command to trigger notifications:

```
python manage.py check_expired_keys
```

Watch the worker terminal — you should see tasks being picked up and executed.

🎓 Instructor Teaching Plan

⏱ Time Breakdown (90 min)

Segment	Duration	Activity
Task queue mental model	15 min	Diagram: producer (management command) → broker (Redis queue) → worker (Celery process) → result backend (Redis). Compare to a restaurant: the kitchen (producer) puts orders on a ticket rail (broker); the cook (worker) picks them up. The worker runs in a separate process — if the web server goes down, tasks already in the queue are safe.
Redis + Celery setup	15 min	Run Redis via Docker. Write `celery.py`, update `__init__.py`. Add broker/backend URLs to `settings.py`. Verify: `celery -A gamekey_platform inspect ping`.
`WebhookDeliveryLog` model	10 min	Add model, migrate, register in admin. Explain: every delivery attempt (success or failure) is a logged row. This is an audit trail for debugging.
Write the Celery task	25 min	Walk through `@shared_task(bind=True, max_retries=3)`. Explain `bind=True` gives us `self` (the task instance). Show `self.retry(exc=exc, countdown=...)`. Explain exponential backoff: 60s → 120s → 240s.
Update management command	10 min	Swap synchronous call for `.delay()`. Explain: `.delay()` puts the task in Redis; it returns immediately. The worker picks it up asynchronously.
Live demo with worker	15 min	Two terminals: one runs the worker, one runs `check_expired_keys`. Students watch tasks appear in the worker log and `WebhookDeliveryLog` entries appear in admin.

💡 Key Concepts to Introduce

- **Task queue** — decouples the producer (who decides what to do) from the worker (who does it). The broker (Redis) is the buffer between them.
- **`@shared_task`** — preferred over `@app.task` because it doesn't require importing the Celery app object. Works with Django's `autodiscover_tasks()`.
- **`bind=True`** — passes the task instance as the first argument (`self`). Required for `self.retry()`.
- **`self.retry(exc, countdown)`** — re-queues the task after `countdown` seconds. Celery tracks the attempt count against `max_retries`.
- **Exponential backoff** — retrying immediately after a failure often fails again (the remote server is still down). Waiting longer each time gives the system time to recover. `60 * (2 ** attempt)` gives 60s, 120s, 240s.
- **`.delay()` vs `.apply_async()`** — `.delay(*args, **kwargs)` is shorthand. `.apply_async(args, kwargs, countdown=30)` gives more control (delays, ETAs, priorities).
- **`CELERY_TASK_ALWAYS_EAGER = True`** — runs tasks synchronously in the same

process. Set this in test settings to avoid needing a real worker in tests.

⚠ Common Mistakes to Address

- **Not importing `celery_app` in `__init__.py`** — Celery's `autodiscover_tasks` won't run. Tasks exist but are never registered with the worker.
- **Running the management command before starting the worker** — tasks queue up in Redis but nothing processes them. Students see `.delay()` return instantly but nothing happens.
- **Iterating the queryset after `.update()`** — `update()` modifies the DB but not the in-memory queryset objects. Always `list()` the queryset *before* calling `.update()` (already shown in the code — make sure students understand why).
- **`self.retry()` must be `raise self.retry()`** — if you call `self.retry()` without `raise`, the function continues executing after the retry call. Always `raise`.
- **Using `apply()` in production** — `apply()` runs the task synchronously (like `ALWAYS_EAGER`). Only for testing. In production, always use `.delay()` or `.apply_async()`.

? Check for Understanding

"What's in Redis right now, after `.delay()` is called but before the worker picks up the task?"

Answer: Redis contains a serialized JSON message (the task payload) representing the Celery task. This message includes the task name, task ID, arguments (like `publisher_id`, `game_key_str`), and other metadata, waiting in a list (acting as a queue).

"What happens if the Celery worker crashes mid-task? Is the task lost?"

Answer: With default settings (late acknowledgment disabled), the task is acknowledged when the worker starts, meaning it could be lost if it crashes mid-execution. If late acknowledgment is enabled (`task_acks_late = True`), the task is only acknowledged *after* successful execution; if the worker crashes, the broker (Redis) will re-queue the task for another worker.

"Why do we log *both* successful and failed deliveries to `WebhookDeliveryLog`?"

Answer: Logging both creates a complete audit trail. It allows developers to diagnose issues, prove to publishers that webhooks were dispatched, analyze error patterns, and keep track of execution history for debugging delivery problems.

"What does `bind=True` do, and why do we need it for retries?"

Answer: `bind=True` binds the task function to the task instance, passing the task instance as the first argument (`self`). We need it because retrying requires calling the `.retry()` method on the task instance itself (`self.retry()`).

"If `max_retries=3` and all retries fail, what happens to the task? How would we know?"

> **Answer:** The task will be marked as `FAILURE` by Celery. An exception (`MaxRetriesExceededError`) will be raised and logged. We would know by checking Celery worker logs, monitoring systems, or checking the `WebhookDeliveryLog` in the database where the final log entry will show `success=False` and the final attempt count.

✓ **Day Checkpoint**

Student runs the Celery worker (`celery -A gamekey_platform worker --loglevel=info`) in one terminal and `python manage.py check_expired_keys` in another. The worker terminal shows task receipt and execution. Admin shows new `WebhookDeliveryLog` entries with `success=True` and `response_status=200`.

Day 9 – Testing with pytest-django

Goals: Write a meaningful test suite covering the expiry flow and webhook dispatch.

```
uv add pytest-django pytest-mock
```

`pytest.ini` (project root):

```
[pytest]
DJANGO_SETTINGS_MODULE = gamekey_platform.settings
python_files = test_*.py
```

`games/tests/__init__.py` — empty file.

`games/tests/test_webhook.py`:

```
import pytest
from django.contrib.auth.models import User
from django.utils import timezone
from datetime import timedelta
from unittest.mock import patch, MagicMock
from django.core.management import call_command
from games.models import Publisher, Game, GameKey, WebhookDeliveryLog


@pytest.fixture
def publisher_user(db):
    user = User.objects.create_user(username='pub_user', password='pass
    publisher = Publisher.objects.create(
        name='Test Publisher',
        webhook_url='https://example.com/webhook',
        webhook_secret='supersecret',
        user=user,
    )
    return publisher


@pytest.fixture
```

```
def expired_game_key(db, publisher_user):
    game = Game.objects.create(
        title='Epic Game',
        publisher=publisher_user,
        price='29.99',
    )
    return GameKey.objects.create(
        key_string='TEST-KEY-0001',
        game=game,
        status='active',
        expires_at=timezone.now() - timedelta(hours=1),
        owner=publisher_user.user,
    )


@pytest.mark.django_db
@patch('games.tasks.send_expiry_webhook_async.delay')
def test_management_command_dispatches_tasks(mock_delay, expired_game_k
    call_command('check_expired_keys')
    assert mock_delay.called
    call_args = mock_delay.call_args
    assert call_args.kwargs['game_key_str'] == 'TEST-KEY-0001'


@pytest.mark.django_db
@patch('games.tasks.requests.post')
def test_webhook_task_logs_success(mock_post, publisher_user, expired_g
    from games.tasks import send_expiry_webhook_async
    mock_response = MagicMock()
    mock_response.status_code = 200
    mock_post.return_value = mock_response

    send_expiry_webhook_async(
        publisher_id=publisher_user.id,
        game_key_str='TEST-KEY-0001',
        game_title='Epic Game',
        expired_at_iso=expired_game_key.expires_at.isoformat(),
        attempt=0,
    )

    log = WebhookDeliveryLog.objects.get(game_key='TEST-KEY-0001')
    assert log.success is True
    assert log.response_status == 200


@pytest.mark.django_db
@patch('games.tasks.requests.post')
def test_webhook_task_logs_failure(mock_post, publisher_user, expired_g
    from games.tasks import send_expiry_webhook_async
    mock_post.side_effect = Exception('Connection refused')

    with pytest.raises(Exception):
```

```
        send_expiry_webhook_async.apply(kwargs=dict(
            publisher_id=publisher_user.id,
            game_key_str='TEST-KEY-0001',
            game_title='Epic Game',
            expired_at_iso=expired_game_key.expires_at.isoformat(),
            attempt=0,
        ))

    log = WebhookDeliveryLog.objects.get(game_key='TEST-KEY-0001')
    assert log.success is False
    assert 'Connection refused' in log.error_message
```

```
pytest --cov=games --cov-report=term-missing
```

🎓 Instructor Teaching Plan

⏱ Time Breakdown (90 min)

Segment	Duration	Activity
Testing philosophy	15 min	What to test and what not to test. Unit tests vs integration tests. The testing pyramid. Why we mock external I/O (network calls, time). What does "70% coverage" actually mean — and what it doesn't mean.
pytest-django setup	10 min	Install `pytest-django` and `pytest-mock`. Write `pytest.ini`. Explain `DJANGO_SETTINGS_MODULE`. Explain `@pytest.mark.django_db` — by default, tests can't access the DB.
Fixtures	15 min	Walk through `publisher_user` and `expired_game_key` fixtures. Explain: fixtures provide reusable setup, run fresh for each test. The `db` fixture enables DB access. Show how fixtures compose (one fixture depends on another).
Walk through the 3 tests	30 min	Test 1: management command dispatches tasks (mock `.delay`). Test 2: task logs success (mock `requests.post`). Test 3: task logs failure (mock raises exception). For each: read the assertion aloud, explain what it proves.
Run coverage + add one more	10 min	Run `pytest --cov=games --cov-report=term-missing`. Read the report together. Challenge students to add a test for the `create_order` view.
Q&A + checkpoint	10 min	

📡 Key Concepts to Introduce

- **Unit vs integration tests** — unit: test one function/method in isolation, mock everything external. Integration: test multiple components working together. Our tests are mostly unit tests with a real DB (pytest-django handles isolation).
- **`@pytest.fixture`** — a function that provides a value to tests. Runs before each test, tears down after. Fixtures can depend on other fixtures.
- **`@pytest.mark.django_db`** — grants DB access for that test. Each test gets a

transaction that's rolled back at the end (no inter-test contamination).

- **`@patch(target)`** — replaces a name in the target module with a `MagicMock` for the duration of the test. The target must be the import path *where the name is used*, not where it's defined.
- **`MagicMock`** — an object that accepts any attribute access or method call and returns another mock. Set `.return_value` to control what it returns, `.side_effect` to make it raise an exception.
- **Coverage** — the percentage of code lines executed by at least one test. 70% is a minimum; it doesn't mean the logic is correct, just that the lines ran.
- **`task.apply(kwargs=dict(...))`** — runs the Celery task synchronously *without* retries. Useful for testing the task body directly without the Celery retry machinery.

⚠ Common Mistakes to Address

- **Wrong `@patch` target** — the most common testing mistake. `@patch('games.tasks.requests.post')` patches `requests.post` *as imported in `games/tasks.py`*. Patching `requests.post` globally won't work if `tasks.py` has already imported it. Rule: patch where it's used.
- **Not using `@pytest.mark.django_db`** — the test appears to pass (no assertion errors) because the fixture never hits the DB, but actually the DB wasn't accessible and the test proved nothing.
- **Calling `.delay()` in tests** — this actually queues the task to Redis (if a broker is running) or raises `OperationalError` (if not). Always use `@patch` or `CELERY_TASK_ALWAYS_EAGER = True` in tests.
- **Not asserting on the right thing** — `assert mock_delay.called` tells you the function was called, but `call_args.kwargs['game_key_str'] == 'TEST-KEY-0001'` verifies the right arguments were passed. Encourage students to assert on the *what*, not just the *whether*.
- **Shared state between tests** — pytest-django wraps each test in a transaction and rolls it back. Students should not rely on data from a previous test existing.

? Check for Understanding

> "Why do we `@patch('games.tasks.requests.post')` rather than `@patch('requests.post')`?"
>
> **Answer:** We patch where the module is imported and used, not where it is defined. In `games/tasks.py`, the code uses `requests.post`. If we patched `requests.post` globally, `games/tasks.py` would still use its local, unpatched reference to the real `requests.post` function.
>
> "What does `mock_post.side_effect = Exception('Connection refused')` do differently from `mock_post.return_value = ...`?"
>
> **Answer:** `.return_value` makes the mock function return a specific value (like a mock response object) when called. `.side_effect` raises the specified exception when the mock function is called, allowing us to test how the code handles connection failures or timeouts.
>
> "If a test has 100% line coverage, does that mean it's bug-free? Why or why not?"

Answer: No, 100% line coverage only means every line of code was executed at least once during the tests. It does not verify different combinations of inputs, edge cases, race conditions, logical errors, or unexpected database states.

"Why do we use `task.apply(kwargs=...)` instead of `task.delay(...)` in the failure test?"

Answer: `task.apply()` runs the task synchronously in the current process, making it easy to catch exceptions and test the task logic step-by-step. `task.delay()` sends the task to Redis asynchronously, making it run in a separate Celery worker process where exceptions are caught by Celery and not raised in the test execution context.

✓ **Day Checkpoint**

`pytest --cov=games --cov-report=term-missing` passes with all 3 tests green and coverage ≥ 70%. Student can explain what each test is verifying and why the mocks are necessary.

Day 10 – Final Integration & Deployment

Goals: Containerise the application and deploy to a cloud provider.

`Dockerfile`:

```
FROM python:3.12-slim

# Install uv
COPY --from=ghcr.io/astral-sh/uv:latest /uv /bin/uv

WORKDIR /app
COPY . .

RUN uv sync --no-dev

CMD ["uv", "run", "python", "manage.py", "runserver", "0.0.0.0:8000"]
```

`docker-compose.yml`:

```
version: '3.9'

services:
  web:
    build: .
    ports:
      - "8000:8000"
    env_file:
      - .env
    depends_on:
      - redis
```

```
    command: uv run python manage.py runserver 0.0.0.0:8000

  worker:
    build: .
    env_file:
      - .env
    depends_on:
      - redis
    command: uv run celery -A gamekey_platform worker --loglevel=info

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
```

```
docker compose up --build
```

Deploying to Render:

1. Push repo to GitHub
2. Create a new **Web Service** on render.com, pointing at your repo
3. Set build command: `uv sync --no-dev && python manage.py migrate`
4. Set start command: `python manage.py runserver 0.0.0.0:$PORT`
5. Add a **Redis** instance (Render provides one) and set `CELERY_BROKER_URL` in environment variables
6. Add a **Background Worker** service using command: `celery -A gamekey_platform worker --loglevel=info`

Final checklist:

- [ ] All endpoints return correct status codes
- [ ] Token auth blocks unauthenticated writes
- [ ] `check_expired_keys` triggers Celery tasks
- [ ] `WebhookDeliveryLog` entries are created per attempt
- [ ] pytest suite passes with ≥70% coverage
- [ ] Docker Compose brings up all three services cleanly

🎓 Instructor Teaching Plan

⏱️ Time Breakdown (90 min)

Segment	Duration	Activity
Docker concepts	20 min	Containers vs VMs. Image = blueprint, container = running instance. Layers (each `RUN` / `COPY` = one layer, cached independently). Why Docker: "works on my machine" problem solved. Multi-service apps need `docker compose`.
Write Dockerfile	15 min	Walk through each instruction. Why `python:3.12-slim` over full Python? Why `WORKDIR /app` before `COPY`? Why `uv sync --no-dev` (no dev dependencies in production)?

Write `docker-compose.yml`	15 min	Three services: `web`, `worker`, `redis`. `depends_on` for startup ordering. `env_file` for environment variables. Separate `command` per service — both `web` and `worker` use the same image but different entry points.
`docker compose up --build`	10 min	Build and run. Test `POST /api/orders/` through the containerized stack. If something fails, read the logs: `docker compose logs web`.
Render deployment walkthrough	20 min	Push to GitHub. Create Web Service, Worker Service, Redis. Set environment variables (especially `CELERY_BROKER_URL`). Trigger a deploy.
Final checklist + wrap-up	10 min	Go through the checklist together. Celebrate what students built. Preview: what would the production hardening look like? (Gunicorn, PostgreSQL, ALLOWED_HOSTS, HTTPS.)

📡 Key Concepts to Introduce

- **Docker image vs container** — image is immutable, like a class definition. Container is a running instance, like an object. Multiple containers can run from the same image.
- **`CMD` vs `RUN`** — `RUN` executes during build (creates image layers). `CMD` runs when a container starts. `docker-compose.yml` `command:` overrides `CMD`.
- **`0.0.0.0` binding** — Django's dev server listens on `127.0.0.1` (loopback) by default. Inside a container, `127.0.0.1` is the container itself — the host machine can't reach it. `0.0.0.0:8000` makes it accessible from outside the container.
- **`env_file` in compose** — loads the `.env` file into the container's environment. Same as setting each variable manually in the `environment:` section but cleaner.
- **`depends_on`** — controls start order, not readiness. Redis might not be fully ready when the web service starts. In production, use health checks (`healthcheck:`) or retry logic.
- **Gunicorn vs `runserver`** — `runserver` is single-threaded, for development only. In production, Gunicorn (or uWSGI) runs multiple worker processes. Render's expected `gunicorn gamekey_platform.wsgi` command vs `runserver`.

⚠️ Common Mistakes to Address

- **`runserver` in production** — Django explicitly warns against this. On Render, use Gunicorn: `gunicorn gamekey_platform.wsgi:application`. Add it with `uv add gunicorn`.
- **`ALLOWED_HOSTS` not updated** — Django rejects requests from unknown hosts in production. Must include the Render domain: `ALLOWED_HOSTS = ['your-app.onrender.com']` or `['*']` temporarily.
- **SQLite in production** — SQLite is a local file. On Render, the filesystem is ephemeral — the DB is wiped on every redeploy. Use Supabase or Neon for PostgreSQL in production.
- **Not running `migrate` in the build command** — deploy succeeds but the app crashes immediately on the first DB request. Build command must be: `uv sync --no-dev && python manage.py migrate`.
- **`CELERY_BROKER_URL` pointing to `localhost`** — inside Docker Compose, services are accessed by service name, not `localhost`. `redis://redis:6379/0` (service name `redis`), not `redis://localhost:6379/0`.
- **Worker service missing** — students deploy only the web service. Webhooks are never

delivered because no Celery worker is running.

? Check for Understanding

"Why do the `web` and `worker` services use the same Docker image but different `command` values?"

Answer: Both services run the same codebase and need the same environment variables, libraries, and settings. However, the `web` service runs the HTTP web server (`manage.py runserver` or Gunicorn) to handle API requests, while the `worker` service runs the Celery daemon (`celery worker`) to process background tasks. Using the same image simplifies building and keeps the dependencies identical.

"What's the problem with binding the Django dev server to `127.0.0.1` inside a Docker container?"

Answer: `127.0.0.1` is the loopback interface, which is only accessible within the container itself. If the dev server is bound to `127.0.0.1`, the host machine and outer network cannot access the container's port. Binding to `0.0.0.0` tells the container to listen on all interfaces, allowing traffic from the host machine to reach the server.

"Why can't we use SQLite in production on Render?"

Answer: SQLite stores data in a local file on the container's disk. Render containers have an ephemeral filesystem, meaning their disks are wiped and recreated on every deployment, restart, or scale event, resulting in complete database loss. Production environments require a persistent, remote database service (like PostgreSQL).

"What would happen if we forgot to add `python manage.py migrate` to the build command?"

Answer: The application would start, but any database queries (such as listing games, user registration, or purchasing keys) would fail with database errors (e.g., `Relation does not exist` or `no such table`) because the database tables would not match the new code models.

✓ **Day Checkpoint (Final)**

`docker compose up --build` starts all three services without errors. `POST /api/orders/` returns `201` through the containerized stack. The Celery worker (in the `worker` container) logs task execution. If deployed to Render: the web service URL returns a valid API response and the worker service shows as "running" in the Render dashboard. `pytest` suite passes with ≥70% coverage.

Project Structure (End of Day 10)

```
gamekey_platform/
├── gamekey_platform/
```

```
│   ├── __init__.py          # Loads Celery app
│   ├── celery.py
│   ├── settings.py
│   └── urls.py
├── games/
│   ├── management/
│   │   └── commands/
│   │       └── check_expired_keys.py
│   ├── migrations/
│   ├── tests/
│   │   └── test_webhook.py
│   ├── admin.py
│   ├── models.py            # Publisher, Game, GameKey, Order, OrderIt
│   ├── permissions.py
│   ├── serializers.py
│   ├── tasks.py             # Celery async webhook task
│   ├── viewsets.py
│   ├── views.py             # register, create_order
│   └── webhooks.py          # sync helper (Day 7 reference)
├── .env
├── .gitignore
├── docker-compose.yml
├── Dockerfile
├── manage.py
├── pytest.ini
└── README.md
```

🔌 API Reference — Sample Requests & Responses

All endpoints are prefixed with `/api/`. Authenticated endpoints require the header:

```
Authorization: Token <your-token>
```

POST `/api/register/`

Register a new user and receive an auth token.

Request

```
POST /api/register/
Content-Type: application/json

{
  "username": "dheeresh",
  "password": "securepass123"
}
```

Response `201 Created`

```
{
```

```
  "token": "9a4f2c8b3e1d6f0a7c5b2e8d4f1a3c7e9b5d2f0a"
}
```

Response `400 Bad Request` — missing fields

```
{
  "error": "Username and password required."
}
```

GET `/api/publishers/`

List all publishers. Public endpoint (no auth needed).

Request

```
GET /api/publishers/
```

Response `200 OK`

```
[
  {
    "id": 1,
    "name": "Valve Corporation",
    "webhook_url": "https://valve.example.com/webhooks/gamekey",
    "user": 2
  },
  {
    "id": 2,
    "name": "CD Projekt Red",
    "webhook_url": "https://cdpr.example.com/hooks/expiry",
    "user": 3
  }
]
```

> `webhook_secret` is always excluded from responses.

POST `/api/publishers/`

Create a publisher profile. Requires auth.

Request

```
POST /api/publishers/
Authorization: Token 9a4f2c8b3e1d6f0a7c5b2e8d4f1a3c7e9b5d2f0a
Content-Type: application/json

{
  "name": "Valve Corporation",
  "webhook_url": "https://valve.example.com/webhooks/gamekey",
  "webhook_secret": "mysecret_abc123",
  "user": 2
```

```
}
```

Response `201 Created`

```
{
  "id": 1,
  "name": "Valve Corporation",
  "webhook_url": "https://valve.example.com/webhooks/gamekey",
  "user": 2
}
```

GET `/api/games/`

List all games. Public endpoint.

Request

```
GET /api/games/
```

Response `200 OK`

```
[
  {
    "id": 1,
    "title": "Half-Life 3",
    "publisher": 1,
    "price": "29.99"
  },
  {
    "id": 2,
    "title": "Cyberpunk 2077",
    "publisher": 2,
    "price": "49.99"
  }
]
```

POST `/api/games/`

Create a game. Requires auth.

Request

```
POST /api/games/
Authorization: Token 9a4f2c8b3e1d6f0a7c5b2e8d4f1a3c7e9b5d2f0a
Content-Type: application/json

{
  "title": "Half-Life 3",
  "publisher": 1,
  "price": "29.99"
}
```

Response `201 Created`

```
{
  "id": 1,
  "title": "Half-Life 3",
  "publisher": 1,
  "price": "29.99"
}
```

GET `/api/games/{id}/`

Retrieve a single game.

Request

```
GET /api/games/1/
```

Response `200 OK`

```
{
  "id": 1,
  "title": "Half-Life 3",
  "publisher": 1,
  "price": "29.99"
}
```

Response `404 Not Found`

```
{
  "detail": "No Game matches the given query."
}
```

PATCH `/api/games/{id}/`

Partially update a game. Requires auth + must be the publisher's owner.

Request

```
PATCH /api/games/1/
Authorization: Token 9a4f2c8b3e1d6f0a7c5b2e8d4f1a3c7e9b5d2f0a
Content-Type: application/json

{
  "price": "19.99"
}
```

Response `200 OK`

```
{
  "id": 1,
  "title": "Half-Life 3",
```

```
  "publisher": 1,
  "price": "19.99"
}
```

Response `403 Forbidden` — wrong owner

```
{
  "detail": "You do not have permission to perform this action."
}
```

POST `/api/orders/`

Buy a game key. Requires auth. Atomically assigns or generates a key with a 30-day expiry.

Request

```
POST /api/orders/
Authorization: Token 9a4f2c8b3e1d6f0a7c5b2e8d4f1a3c7e9b5d2f0a
Content-Type: application/json

{
  "game_id": 1
}
```

Response `201 Created`

```
{
  "order_id": 42,
  "game": "Half-Life 3",
  "key": "A3F9-B21C-4DE7-9901-CC84",
  "expires_at": "2026-07-16T10:30:00Z"
}
```

Response `400 Bad Request` — missing `game_id`

```
{
  "error": "game_id is required."
}
```

Response `404 Not Found` — game doesn't exist

```
{
  "error": "Game not found."
}
```

Response `401 Unauthorized` — no token provided

```
{
  "detail": "Authentication credentials were not provided."
}
```

Webhook Payload — `game_key.expired`

Sent asynchronously by Celery to the publisher's `webhook_url` when a key expires. Signed with HMAC-SHA256.

Headers

```
Content-Type: application/json
X-Signature: sha256=3c6e9b52a3c2ac3f7dca0c944b6d68b3f15ea2e3c0cf1b2e5f7
```

Body

```
{
  "event": "game_key.expired",
  "game_key": "A3F9-B21C-4DE7-9901-CC84",
  "game_title": "Half-Life 3",
  "expired_at": "2026-07-16T10:30:00Z",
  "attempt": 0
}
```

`attempt` increments on each retry (max 3). Verify the signature on your end:

```
import hmac, hashlib, json
expected = hmac.new(secret.encode(), json.dumps(payload, sort_keys
assert hmac.compare_digest(f"sha256={expected}", request.headers['
```

Webhook Retry Behaviour

Attempt	Delay before retry
0 → 1	60 seconds
1 → 2	120 seconds
2 → 3	240 seconds
3	Task marked failed, logged

All attempts (success or failure) are recorded in `WebhookDeliveryLog`.

Error Reference

Status	Meaning
`200 OK`	Successful read
`201 Created`	Resource created
`400 Bad Request`	Missing or invalid input
`401 Unauthorized`	No or invalid token
`403 Forbidden`	Authenticated but not the owner
`404 Not Found`	Resource does not exist

---

10 days · 1.5 hours/day · production-ready Django API with async webhooks